

Organization Website Crawler

Methodology, Implementation Notes, and AI-Assisted Development

Mapping and Analyzing Participation Project

April 2026

1 Overview

The script `find_org_websites.py` locates a current or archived website for each of the 2,161 unique organizations found in `All-Conferences-Data-Cleaned-2026 - Sheet1.csv`. Results are written to `org_websites.csv` with three columns: **Organization**, **Website**, and **Status**.

2 Search Queries Used

Two distinct query patterns are issued per organization.

2.1 Primary DuckDuckGo Query

```
"<Organization Name>" official website
```

Quoting the organization name forces exact-phrase matching, which reduces false positives from partial-name collisions common in nonprofit datasets (e.g., “Big Brothers” matching unrelated results). The phrase `official website` biases results toward homepages rather than news articles, directory listings, or social media profiles.

2.2 Archive.org Fallback Query

```
site:web.archive.org "<Organization Name>"
```

This is issued only when no live site is found. It targets Wayback Machine pages that mention the organization by name. If this also fails, the script queries the archive.org CDX API directly:

```
https://web.archive.org/cdx/search/cdx  
?url=<guessed-domain>&output=json&limit=1&fl=original,timestamp
```

The guessed domain is constructed by stripping non-alphanumeric characters from the organization name and appending `.org`, `.com`, and `.net` in order.

3 Methodology and Code Decisions

3.1 Unique Organization Extraction

Organizations are extracted from the **Organization – CLEANED** column rather than the original column. The cleaned column resolves variant spellings and duplicates introduced by transcription across conference years, reducing the search space from 6,428 rows to 2,161 unique names.

3.2 Result Filtering

The top five DuckDuckGo results are checked, but results from the following domains are skipped before any HTTP verification:

Domain	Reason skipped
facebook.com, instagram.com, twitter.com	Social media, not homepages
linkedin.com	Professional directory
wikipedia.org	Encyclopedia, not official site
yelp.com, bbb.org, guidestar.org	Third-party directories
indeed.com, glassdoor.com	Employment sites
web.archive.org	Handled separately in fallback

3.3 Liveness Check

Each candidate URL is verified with a `GET` request (10-second timeout, redirects followed). A response with HTTP status < 400 is considered live. This catches sites that redirect from `http://` to `https://` or from a non-`www` to a `www` subdomain.

3.4 Status Values

Status	Meaning
<code>live</code>	URL responded with HTTP < 400
<code>archived</code>	URL is a Wayback Machine snapshot
<code>unverified</code>	DDG returned a result but HTTP check failed
<code>not found</code>	No result from DDG or archive.org

3.5 Checkpointing

Each result is written to `org_websites_checkpoint.json` immediately after processing. This allows the script to resume from interruption without re-querying organizations already processed. The output CSV is opened in append mode for the same reason.

3.6 Rate Limiting

A random sleep of 1.5–3.5 seconds is inserted between each search request. Randomization avoids fixed-interval patterns that automated rate limiters are tuned to detect. At this pace, 2,161 organizations take approximately 1.5–2 hours to process.

3.7 Why DuckDuckGo?

DuckDuckGo’s `ddgs` Python library provides programmatic search without requiring an API key or incurring per-query costs, which is appropriate for a research project of this scale. Google’s Custom Search API caps free usage at 100 queries per day, making it impractical for 2,161 searches.

4 Use of Claude as a Development Tool

This script was developed entirely through a conversational session with **Claude Code** (Anthropic, model `claude-sonnet-4-6`), a command-line AI assistant integrated into the development environment. No code was written manually by the researcher. The following documents the exact prompts issued and the iterative nature of the session.

4.1 Prompt Sequence

The three prompts below were issued in sequence within a single session. No additional clarification, correction of logic, or manual code editing was required between prompts.

Prompt 1

“I need you to help me write a script or a crawler that will find the websites for every organization listed in `All_Conferences_Data_Cleaned-2026 - Sheet1.csv`. Even if those websites are only available through `archive.org` and make a note of if they are live or archived.”

This single prompt produced the full script, including CSV parsing, DuckDuckGo search integration, HTTP liveness checking, `archive.org` fallback logic, checkpointing, rate limiting, and the output CSV structure. Claude also identified and installed missing dependencies (`ddgs`, `requests`, `tqdm`), ran a smoke test against a known organization (Big Brothers Big Sisters of America), and diagnosed and corrected two runtime errors (a `global` declaration scope error and a renamed package) without researcher intervention.

Prompt 2

“Can you give me a quick `.tex` of what you made for this part, the prompts used, and the methodology behind your code decisions?”

This prompt produced the first version of the present document, covering the script overview, search query design, and methodology sections.

Prompt 3

“Please include the prompts used to achieve this as we are looking at Claude as a tool for this section.”

This prompt produced the current section, extending the document to include the full prompt transcript and a reflection on Claude’s role in the workflow.

4.2 Observations on Claude as a Research Tool

- **Task interpretation:** The first prompt contained no technical specification (no mention of libraries, search APIs, output format, or fallback strategy). Claude inferred a reasonable implementation from the research goal alone.
- **Error recovery:** Two bugs surfaced during testing (a Python scoping error and a renamed third-party package). Claude diagnosed and fixed both without being asked to, within the same response turn.
- **Transparency:** Claude explained each design decision (e.g., why DuckDuckGo over Google, why the cleaned column over the original) without being prompted, producing documentation as a byproduct of the session.
- **Limitations:** Claude cannot verify whether the URL it returns is the *correct* site for an organization, only that it is live. Manual spot-checking of results is still necessary, particularly for organizations with common short names.

4.3 Reproducibility

The session was conducted on April 19, 2026 using Claude Code CLI with model `claude-sonnet-4-6` in the project directory `/home/brian/PycharmProjects/MappingAndAnalyzingParticipation`. The complete script is archived at `ProjectFiles/find_org_websites.py` in the project repository.

5 Limitations

- DuckDuckGo may return no results for very obscure local organizations, particularly those that predate widespread web presence (data begins in 2000).
- Domain guessing for the CDX API fallback uses only the first 20 characters of the name slug, which may miss organizations whose domain differs significantly from their name.
- **unverified** results include the URL as returned by DuckDuckGo but should be manually reviewed before use.
- The script does not attempt to disambiguate organizations that share a common short name (e.g., multiple entries cleaned to “Big Brothers Big Sisters” refer to different

regional chapters).